

Parte 1. Práctica 1.

Lourdes Domínguez

13 de septiembre de 2025

Resumen

En esta práctica aplicaremos el método de Monte Carlo para estimar integrales y calcular el volumen de una N-hiperesfera, analizando cómo influyen el número de iteraciones y la elección del máximo en la precisión de los resultados.

1. Parte 1. Obligatoria.

En esta práctica se ha visto que el método de Monte Carlo permite obtener una buena estimación de la integral de la función:

$$f(x) = \sin^2\left(\frac{1}{x(2-x)}\right) \quad (1)$$

en el intervalo $[0, 2]$, siempre que el máximo elegido para acotar la función sea adecuado y haya suficientes puntos de muestreo. A lo largo de esta sección analizaremos la efectividad del método al variar esos factores.

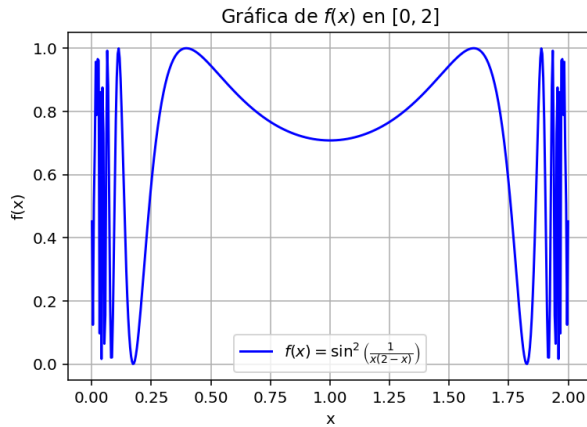


Figura 1: Gráfica de la función pedida.

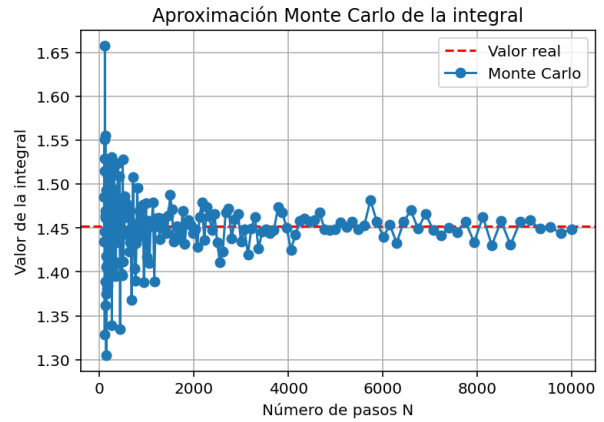


Figura 2: Aproximación del valor de la integral con Monte Carlo mediante el aumento del número de iteraciones.

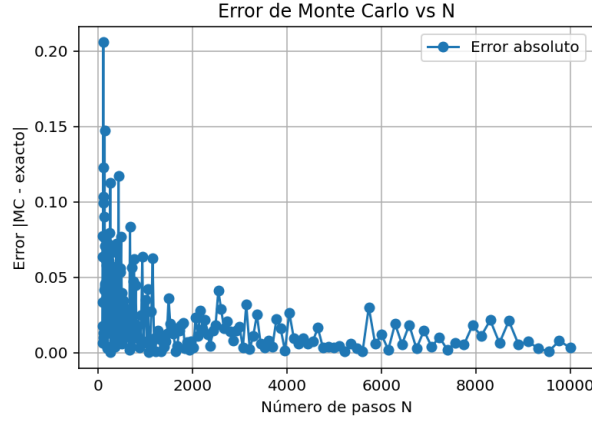


Figura 3: Error de la aproximación de la integral con Monte Carlo mediante el aumento del número de iteraciones.

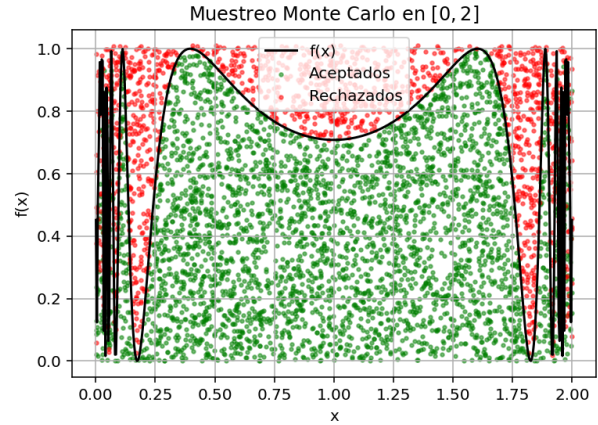


Figura 4: Función con los puntos que caen dentro y fuera de la función.

Comenzaremos muestreando la gráfica de la función (figura 6) para poder visualizar donde está el máximo y cómo se comporta la función en el intervalo.

A continuación, se ha implementado el método de Monte Carlo para estimar la integral de la función, variando el número de iteraciones (figura 7) y observando cómo la estimación converge al valor real de la integral a medida que aumenta el número de puntos muestreados. Además, como veremos a continuación, se ha observado que la eficiencia del método depende de cómo se elige el máximo y el intervalo de muestreo, lo veremos también en la figura 5.

La figura 10 muestra cómo el error en la estimación disminuye con el aumento del número de iteraciones, ilustrando así la eficiencia del método. Explicaremos más detalladamente esta dependencia a continuación.

Finalmente, la figura 11 muestra los puntos generados aleatoriamente, diferenciando los que caen dentro y fuera de la función. Esta visualización ilustra cómo el método Monte Carlo reproduce un proceso físico de muestreo aleatorio estimando así la integral: los puntos bajo la curva representan eventos aceptados y su proporción respecto al total aproxima el área buscada. Cabe destacar la importancia de este procedimiento, ya que conecta con la interpretación probabilística de la integral y emplea el mismo método que experimentos importantes como la detección de partículas en física nuclear.

Ahora vamos a analizar la dependencia del método montecarlo con el número de iteraciones y la elección del máximo. Asimismo explicaremos la disminución del error con el aumento del número de iteraciones.

Dependencia con el máximo:

Cuando el valor del máximo de la función se escoge por debajo del verdadero, el método introduce un sesgo sistemático: obtenemos un valor de la integral menor del esperado porque parte del área bajo la curva queda fuera del rectángulo de muestreo y, por tanto, no se contabiliza en el cálculo. Esto significa que, aunque aumentemos el número de pasos, la estimación converge a un valor incorrecto, como se observa en la gráfica 5 donde convergería a un valor menor de 1,4 cuando el valor es de aproximadamente 1,45. Como vamos cuando N tiende a grandes números el resultado tiende a estabilizarse, pero no representa el valor real de la integral, ya que la región excluida nunca será muestreada.

Por otro lado, si se elige un máximo mucho mayor que el real, el método sigue siendo matemáticamente válido y no introduce sesgo, pero se vuelve mucho menos eficiente. En este caso, la mayoría de los puntos generados aleatoriamente quedan fuera de la función y son rechazados, lo que provoca que la estimación sea más variable y requiera un número mucho mayor de iteraciones para alcanzar un valor parecido al real. Esto se traduce en un mayor coste computacional y en una convergencia más lenta, ya que el número de puntos aceptados respecto al total disminuye considerablemente.

En resumen, la elección adecuada del máximo es fundamental para obtener estimaciones precisas y eficientes con el método Monte Carlo. Un máximo demasiado bajo introduce sesgo y subestima la integral, mientras

que un máximo demasiado alto reduce la eficiencia y ralentiza la convergencia. Por ello, es importante analizar la función previamente y seleccionar un máximo lo más ajustado posible al valor real.

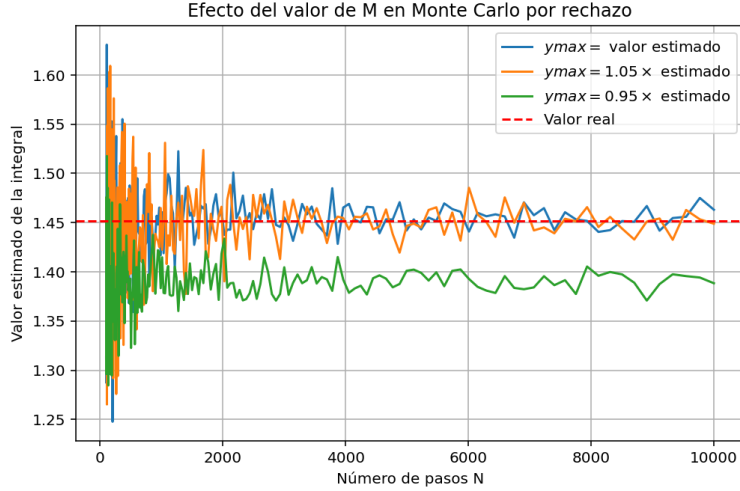


Figura 5: Comparación de la aproximación del valor de la integral con Monte Carlo mediante el aumento del número de iteraciones para diferentes máximos.

Dependencia con el número de pasos y error del método:

El valor estimado de la integral mediante el método Monte Carlo depende directamente del número de puntos de muestreo, N . Al principio, cuando N es pequeño, la gráfica de la estimación muestra valores dispersos y fluctuaciones notables, reflejando la incertidumbre del muestreo aleatorio. Sin embargo, a medida que se incrementa el número de puntos, la estimación comienza a estabilizarse y se aproxima progresivamente al valor real de la integral.

Este comportamiento ilustra de manera visual la ley de los grandes números: con suficientes muestras, la media de los resultados converge hacia el valor esperado, igual que ocurre en mediciones experimentales donde la precisión mejora al aumentar el número de datos. Así, la gráfica no solo muestra la convergencia del método Monte Carlo, sino que también permite observar cómo las fluctuaciones estadísticas disminuyen con el número de iteraciones.

Matemáticamente, el error estadístico asociado a la estimación de la integral se reduce aproximadamente como $1/\sqrt{N}$, según la expresión:

$$\sigma = \sqrt{\frac{\text{Var}[f(x)]}{N}} \quad (2)$$

Esto implica que, al aumentar N (como vemos en la figura 10), la estimación se estabiliza alrededor del valor verdadero y las barras de error se estrechan, mejorando la precisión del resultado. Visualmente, al representar los puntos aceptados junto con la función, se aprecia que con pocos pasos la nube de puntos es dispersa y apenas reproduce la forma de la curva, mientras que al aumentar N la región bajo la función se rellena y la estimación converge de manera clara hacia el valor exacto.

2. Parte 2. Resultados con la nueva función.

$$f(x) = \frac{e^{x^2}}{(x - 24)^2 + 0,024} \quad (3)$$

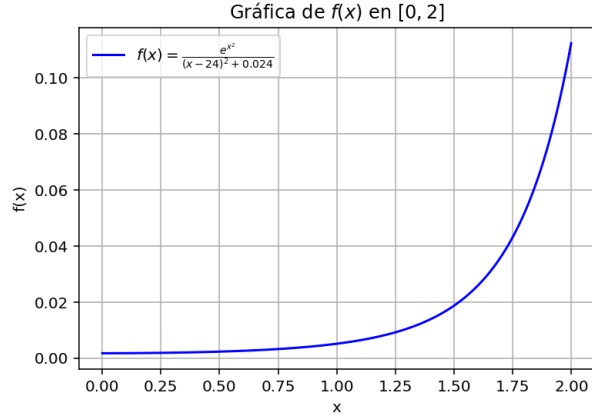


Figura 6: Gráfica de la función nueva.

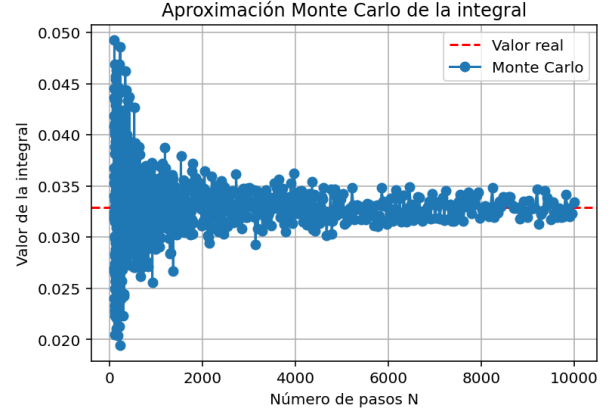


Figura 7: Aproximación del valor de la integral de la función nueva con Monte Carlo mediante el aumento del número de iteraciones.

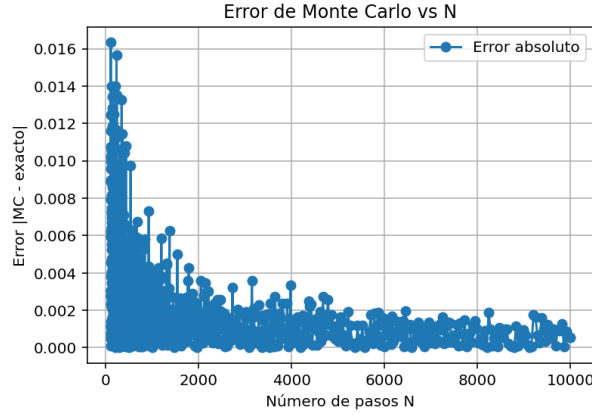


Figura 8: Error de la aproximación de la integral de la función nueva con Monte Carlo mediante el aumento del número de iteraciones.

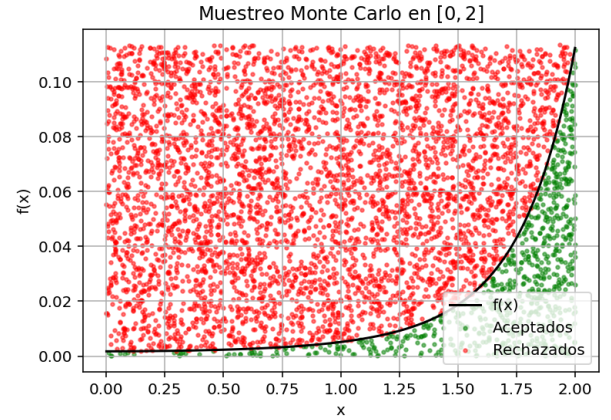


Figura 9: Función nueva con los puntos que caen dentro y fuera de la función.

Se obtienen resultados análogos a los anteriores: la estimación de la integral converge al valor real al aumentar el número de iteraciones, el error disminuye como $1/\sqrt{N}$ y la eficiencia depende de la elección adecuada del máximo, mostrando la robustez y las mismas características del método observadas previamente.

3. Parte 3. Volumen de una N-hiperesfera.

El volumen de una N-hiperesfera de radio r en $\mathbb{R}^N$ está dado por la fórmula:

$$V_N(r) = \frac{\pi^{N/2}}{\Gamma(N/2 + 1)} r^N \quad (4)$$

Sin embargo, si no conocemos la expresión real del volumen este se puede aproximar por el método Monte Carlo. Para ello, se genera un conjunto de puntos aleatorios en el espacio $\mathbb{R}^N$ y se determina cuántos de estos puntos caen dentro de la hiperesfera (de forma similar a lo que hicimos en la primera parte). La relación entre el número de puntos dentro y fuera de la hiperesfera permite estimar su volumen.

Para aproximar este volumen mediante el método de Monte Carlo, consideramos el siguiente razonamiento: El volumen de la hiperesfera es igual al volumen del hiper-cubo que la contiene multiplicado por la

probabilidad de que un punto aleatorio dentro del hiper-cubo caiga dentro de la hiperesfera. Matemáticamente, si generamos N puntos aleatorios en el hiper-cubo de lado $2r$, el volumen de la hiperesfera se estima como:

$$V_{\text{esfera}} \approx V_{\text{cubo}} \cdot \frac{N_{\text{dentro}}}{N} \quad (5)$$

donde $V_{\text{cubo}} = (2r)^N$ es el volumen del hiper-cubo y N_{dentro} es el número de puntos que cumplen $x_1^2 + x_2^2 + \dots + x_N^2 \leq r^2$. Con lo que si lo introducimos en el código quedará de la siguiente manera:

```

1 def dentro_esfera(punto, radio):
2     # Calculamos la suma de los cuadrados de las coordenadas
3     # Si es menor o igual que radio al cuadrado, el punto esta dentro
4     return sum(x**2 for x in punto) <= radio**2
5
6     # -----
7     # Funcion que calcula el volumen mediante Monte Carlo
8 def volumen_MC(dimensiones, radio, N):
9     dentro = 0 # Contador de puntos que caen dentro de la hiperesfera
10
11     for _ in range(N):
12         # Generamos un punto aleatorio dentro del hiper-cubo [-radio, radio]^
13         dimensiones
14         punto = [uniform(-radio, radio) for i in range(dimensiones)]
15
16         # Miramos si el punto esta dentro de la hiperesfera
17         if dentro_esfera(punto, radio):
18             dentro += 1 # Incrementamos el contador si esta dentro
19
20     # Calculamos el volumen del hiper-cubo que contiene la hiperesfera
21     V_cubo = (2 * radio)**dimensiones
22
23     # La fraccion de puntos dentro de la esfera multiplicada por el volumen del
24     hiper-cubo
25     #nos da una estimacion del volumen de la hiperesfera
26     V_esfera = (dentro / N) * V_cubo
27
28     return V_esfera

```

Para poder visualizar los resultados obtenidos, se han representado gráficamente el volumen de la hiperesfera y el error relativo de la aproximación Monte Carlo respecto al número de dimensiones (figuras 10 y 11).

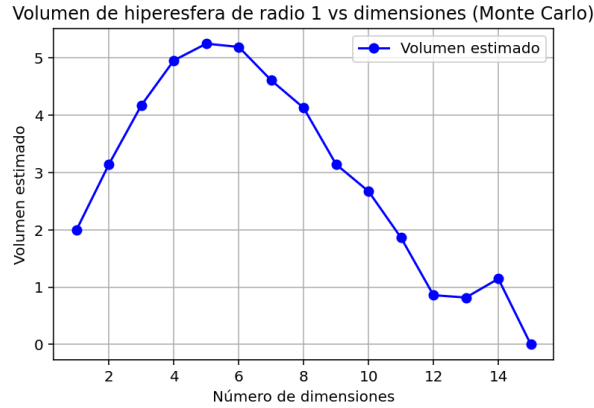


Figura 10: Volumen de la N-hiperesfera respecto al número de dimensiones.

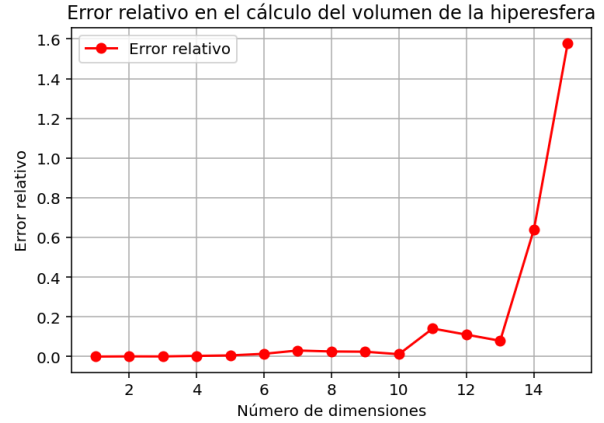


Figura 11: Error relativo del volumen de la N-hiperesfera respecto al número de dimensiones.

Los valores obtenidos mediante el método comparados con los reales serían:

Dimensiones (N)	Volumen Exacto	Volumen Aproximado (Monte Carlo)	Error absoluto
2	3.1416	3.14572	0.00412
3	4.1888	4.20448	0.01568
4	4.9348	4.91616	0.01864
5	5.2638	5.26368	0.00012
6	5.1677	5.18144	0.01374
7	4.7248	4.61056	0.11424
8	4.0587	4.03968	0.01902
9	3.2985	3.21024	0.08826
10	2.5502	2.2016	0.34860
11	1.8841	1.7408	0.14330
12	1.3353	1.35168	0.01638
13	0.9106	1.06496	0.15436
14	0.5993	0.8192	0.21990
15	0.3810	0.32768	0.05332

Cuadro 1: Volumen de la N-hiperesfera en diferentes dimensiones y error absoluto de la aproximación Monte Carlo.

Como muestran la tabla y la figura, el método de Monte Carlo proporciona una buena aproximación al volumen exacto de la N-hiperesfera para dimensiones bajas y medias. En particular, para $N = 2$ y $N = 3$, el volumen estimado es muy cercano al valor real, mostrando la eficiencia del método en estos casos. El error absoluto es pequeño para $N \leq 6$, pero a medida que aumenta la dimensión, el error tiende a crecer. Para dimensiones superiores ($N > 10$), el error absoluto puede ser significativo, lo que indica que sería necesario aumentar considerablemente el número de muestras para mantener la calidad de la aproximación en dimensiones altas.

En resumen, el método Monte Carlo es una herramienta para calcular volúmenes cuando no existe una fórmula analítica sencilla pero su precisión depende fuertemente del número de puntos utilizados y de la dimensión del espacio considerado.

4. Cambios en el código.

Mencionaré en un primer momento los cambios realizados en el código de la primera parte. Para después indicar cómo se ha creado el código de la tercera parte saltandome la explicación ya indicada en el apartado

anterior.

4.1. Primera parte.

Hemos añadido una sección donde se nos grafica la dependencia de la convergencia del método Monte Carlo con el máximo elegido. Anteriormente no lo había incluido ya que no se menciona la necesidad de crearla pero creo que ayuda a entender mejor la dependencia del método con el máximo. El código es el siguiente:

```
1  # -----
2  # 5. Comparacion con diferentes valores de ymax
3  ymax_centro = ymax    # maximo real
4  ymax_mayor  = ymax * 1.15 # aumentamos un 15 por ciento el valor del maximo
5  ymax_menor  = ymax * 0.85 # disminuimos un 15 por ciento el valor del maximo
6
7  val_MC_centro, val_MC_mayor, val_MC_menor = [], [], [] # creamos listas para
   guardar valores
8  Npasos = np.logspace(2, 4, 200, dtype=int) # numero de pasos que tendra nuestra
   iteracion
9
10 for N in Npasos:
11     val_c = integral_MC(a, b, ymax_centro, N)[0] # obtenemos valores con ymax
   esperado
12     val_mas = integral_MC(a, b, ymax_mayor, N)[0] # obtenemos valores con ymax
   mayor
13     val_menos = integral_MC(a, b, ymax_menor, N)[0] # obtenemos valores con ymax
   menor
14
15     # guardamos en la lista para graficar
16     val_MC_centro.append(val_c)
17     val_MC_mayor.append(val_mas)
18     val_MC_menor.append(val_menos)
19
20 # -----
21 # Grafica comparativa
22 plt.figure(figsize=(8,5))
23 plt.plot(Npasos, val_MC_centro, '-', label=r'$ymax = $ valor estimado')
24 plt.plot(Npasos, val_MC_mayor, '-', label=r'$ymax= 1.05 \times$ estimado')
25 plt.plot(Npasos, val_MC_menor, '-', label=r'$ymax = 0.95 \times$ estimado')
26 plt.axhline(valor_real, color='r', linestyle='--', label="Valor real")
27
28 plt.xlabel("Numero de pasos N")
29 plt.ylabel("Valor estimado de la integral")
30 plt.title("Efecto del valor de M en Monte Carlo por rechazo")
31 plt.legend()
32 plt.grid(True, which="both")
33 plt.show()
```

4.2. Tercera parte.

Tras el código ya mostrado en el apartado 3 primero se han calculado los valores específicos de la siguiente forma:

```
1  radio = 1
2  dimensiones = 2
3  N = 100000 # Numero de puntos Monte Carlo
4
5  vol2D = volumen_MC(dimensiones, radio, N)
```

```

6 print(f"Area aproximada del circulo (2D, radio={radio}): {vol2D:.6f} ($\pi$ $\backslash$
    approx$ {np.pi:.6f})")

```

Tras ello lo que se hizo es un bucle para calcular el volumen en diferentes dimensiones y graficar los resultados. El código es el siguiente:

```

1  # -----
2  # Ahora vamos a ver como depende del numero de dimension
3  radio = 1
4  N = 100000 # numero de puntos Monte Carlo
5  dimensiones_max = 15
6  dimensiones_array = np.arange(1, dimensiones_max + 1) # de 1 a 15 dimensiones
7  valores_MC = []
8
9  # -----
10 # Calculamos el volumen estimado para cada dimension
11 for dim in dimensiones_array:
12     vol = volumen_MC(dim, radio, N)
13     valores_MC.append(vol)
14 print(valores_MC)
15 # -----
16 # Graficamos el volumen estimado vs numero de dimensiones
17 plt.figure()
18 plt.plot(dimensiones_array, valores_MC, 'o-', color='blue', label="Volumen estimado
    ")
19 plt.xlabel("Numero de dimensiones")
20 plt.ylabel("Volumen estimado")
21 plt.title("Volumen de hiperesfera de radio 1 vs dimensiones (Monte Carlo)")
22 plt.grid(True)
23 plt.legend()
24 plt.show()

```

Finalmente, empleamos la ecuación analítica del volumen de la N-hiperesfera (ecuación 4) para calcular el error relativo de la aproximación Monte Carlo y graficarlo. El procedimiento es el mismo que el llevado a cabo en la primera parte solo que en vez de depender del número de iteraciones lo obtenemos en función del número de dimensiones. El código es el siguiente:

```

1  # -----
2  # Calculo del volumen exacto con formula analitica
3  def volumen_real(dimensiones, radio):
4      return (np.pi**((dimensiones/2) / gamma(dimensiones/2 + 1)) * (radio**
        dimensiones))
5
6  # -----
7  # Calculamos los errores relativos para cada dimension
8  valores_reales = [volumen_real(dim, radio) for dim in dimensiones_array]
9  errores_relativos = [abs((mc - real) / real) for mc, real in zip(valores_MC,
    valores_reales)]
10
11 # -----
12 # Graficamos el error relativo vs numero de dimensiones
13 plt.figure()
14 plt.plot(dimensiones_array, errores_relativos, 'o-r', label="Error relativo")
15 plt.xlabel("Numero de dimensiones")
16 plt.ylabel("Error relativo")
17 plt.title("Error relativo en el calculo del volumen de la hiperesfera")
18 plt.grid(True)
19 plt.legend()
20 plt.show()

```